ALL ABOUT AGENTS · SYSTEM DESIGN · PROGRESSIVE BUILD

A Mental Model for Designing Agentic Systems

Starting from three black-box wishes and growing them—question by question—into a reliable hybrid architecture.

By Eva Khmelinskaya · November 2026 · about 15 minutes to read

It is tempting to treat an AI agent as a genie in a bottle: you state a wish, and the result appears. As frontier models improve and earn trust in everyday work, handing them a vague goal and watching them figure out the steps becomes routine. The more capable they get, the easier it is to carry that habit into system design and draw each agent as an autonomous black box that just works.

At the same time, the tooling around these models moves fast. New frameworks, SDKs, and APIs keep arriving, and there is always another interface to learn. Underneath that churn, the architectural questions change far more slowly. Like Episode 01, this episode starts with the simplest possible diagram and grows it across six stages—adding only the boxes and boundaries that twelve design questions force onto the page.

Design Scenario

This episode is framed around a twenty-minute design exercise—intentionally tight, so you focus on structural invariants rather than getting lost in framework details.

> **Scenario:** Design a self-healing code repository system with three agents: an agent that detects issues, an agent that suggests a fix, and an agent that opens a pull request with the fix. It is expected to handle `10,000` deployments a day across `20` to `30` repositories, of which `1%` fail.

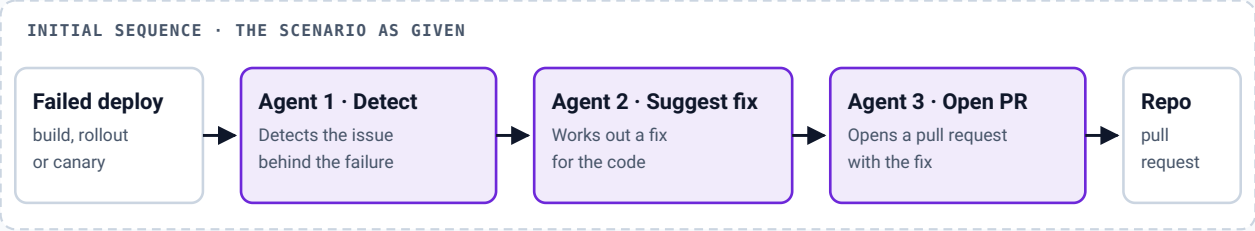


Figure 1 — Initial sequence of agents from the scenario. ▢ Unexamined black-box agent

> **Challenge yourself before reading on.** Take a few minutes and look at *Figure 1*. Which steps genuinely need a language model, what is missing around those three boxes, and what questions would you want answered before calling the design finished?

Why Use a Language Model at All—and Why Wrap It in Code

Before adding a single box to *Figure 1*, ask why plain deterministic software cannot solve the problem on its own. Classical code excels whenever inputs arrive in fixed schemas and every transformation from input to action can be written ahead of time as explicit rules. It breaks down at two boundaries that recur across domains: **interpreting unstructured, noisy inputs** whose phrasing and format vary too widely for static parsers or regular expressions, and **exploring an open-ended solution space** where the next lookup or candidate change depends on what the previous step uncovered. Language models earn their place in an architecture at those two points, because non-deterministic reasoning generalizes across unfamiliar inputs and synthesizes candidate actions that could never be enumerated in advance as rules.

At the same time, the exact property that gives a model that reach—probabilistic generation over open-ended context—makes it the wrong mechanism for system

invariants such as deduplication, storm limits, lease ownership, self-grading, or holding write credentials. Production systems combine both in a **hybrid architecture** that embeds scoped **agents** inside deterministic **workflows** (Anthropic; Google Cloud; Cognition; 12-Factor Agents). That split rests on a general asymmetry: *proposing* a structured interpretation or candidate fix from messy context is hard to write as rules, whereas *verifying* whether a candidate output satisfies schema, policy, and test suites—and committing the side effect once—is exact in deterministic code. Let us now grow *Figure 1* across six stages.

STAGE 1 OF 6 · QUESTIONS 1–3 · FRAMING & FAST CONTAINMENT

Separate Urgent Containment from Asynchronous Agent Work

Define what wakes the system up, check the real event rate, and take live bleeding off the agent path.

WHERE WE ARE

Figure 1 starts at "Failed deploy" as if every failure arrives in one format and production can wait minutes while an agent reads logs and runs tests. Answering Questions 1 to 3 adds normalized triggers and a separate, model-free fast path in *Figure 2*.

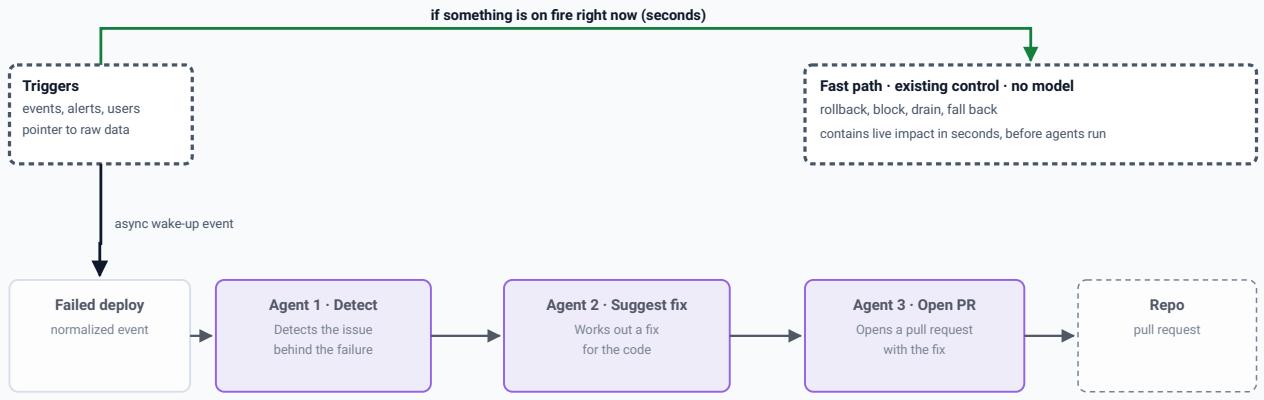


Figure 2 — Stage 1 of 6 (Q1–Q3): Normalized triggers and a model-free fast path added above the initial sequence.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

1 What wakes the system up, and what is in the wake-up message?

SKIP IT AND…

- Two of the three failure sources never trigger the workflow.
- 50 MB of raw logs get stuffed onto the event bus.

DECIDE

- List every upstream trigger source up front.
- Normalize all of them into one small event carrying a **pointer** to raw data, not the raw payload itself.

IN THE SELF-HEALING REPO

- CI failure, canary rollout alert, and error-rate alarm.
- `FailureEvent(repo, commit_sha, failure_fingerprint, log_pointer)`.

2 How big is it really?

SKIP IT AND…

- You over-engineer sharded caches for `10,000`/day when only `100`/day run.
- Or a hidden retry loop multiplies model spend by 50×.

DECIDE

- Multiply total volume by the filter rate before choosing infrastructure.
- Ask whether you face a throughput problem or a correctness/coordination problem, and cap cost per case.

IN THE SELF-HEALING REPO

- `10,000 × 1% = 100` failures/day (~one every 14 minutes)—a correctness problem, not a QPS race.
- Hard step cap and model budget per incident.

3 What must happen in seconds, and who does it?

SKIP IT AND…

- Users suffer a broken release for ten minutes while a model deliberates.
- Or an unreviewed model pushes a hotfix straight to production.

DECIDE

- Split **containment** (seconds, deterministic, existing automation) from **diagnosis and repair** (minutes, asynchronous).
- Agents never sit in the live emergency path.

IN THE SELF-HEALING REPO

- The deployment controller rolls back the failing canary in seconds without calling a model.
- Agents work afterward on a branch to prepare a draft PR.

Added in Stage 1: Normalized pointer triggers + model-free fast containment path

Still open: Caller timeouts & dropped bursts · duplicate runs & storms · black-box agents · handoffs · sandboxes

Live traffic is now safe while the workflow takes minutes—yet upstream callers still post directly to the worker and lose events on bursts or crashes. Stage 2 fixes intake.

STAGE 2 OF 6 · QUESTION 4 · RELIABLE INTAKE & TERMINAL ACK

Decouple Callers from Slow Runs with a Receiver and Queue

Authenticate fast at the edge, buffer work durably, and acknowledge only when a run finishes.

WHERE WE ARE

Webhooks from CI or monitoring time out after a few seconds, whereas testing a candidate fix takes minutes. Inserting a stateless receiver, a durable queue, and a terminal acknowledgement (`ack`) path in *Figure 3* prevents dropped events.

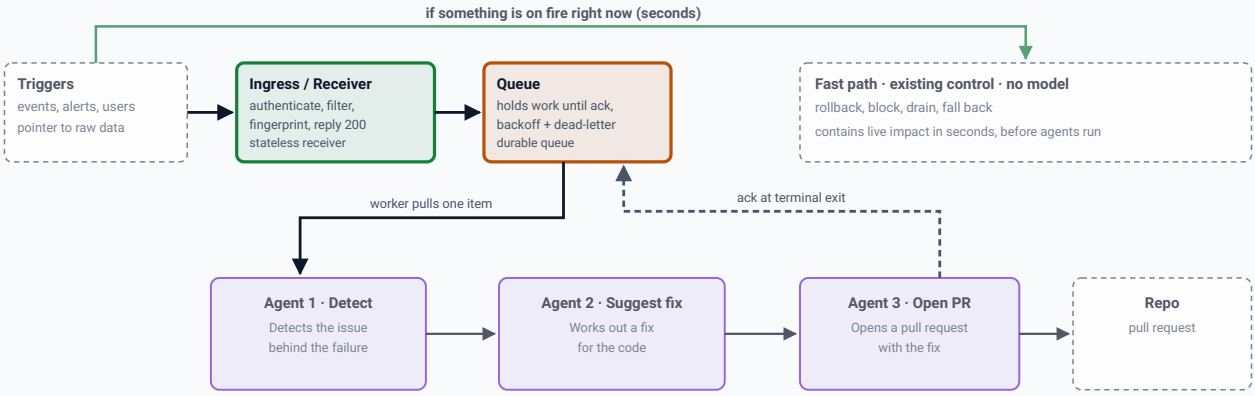


Figure 3 — Stage 2 of 6 (Q4): Stateless receiver, durable queue, and terminal ack decouple callers from slow runs.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

4 How does work get from the event to a worker without being lost?

SKIP IT AND…

- The webhook sender times out waiting for a slow model run.
- A burst drops events, or an unverified endpoint accepts spoofed webhooks.

DECIDE

- Place a stateless **Receiver** in front: verify sender signature/token, drop healthy noise, reply `200` at once.
- Buffer in a durable **Queue**; send `ack` only at a terminal exit, with exponential backoff and a dead-letter queue after N crashes.

IN THE SELF-HEALING REPO

- Webhook receiver → Pub/Sub → worker.
- HMAC/OIDC check, reply in <100 ms, `ack` at `PR_OPENED` / `SKIP` / `ESCALATED`, DLQ after 5 crashes.

Added in Stage 2: Stateless Ingress / Receiver + durable Queue + terminal ack

Still open: Duplicate deliveries · multi-repo outage storms · dead or stalled workers · black-box agents

At-least-once queueing guarantees no event is lost, which immediately creates the opposite hazard: the same failure delivered twice, a shared outage flooding every repo, or a worker dying mid-run. Stage 3 adds coordination.

STAGE 3 OF 6 · QUESTIONS 5–7 · COORDINATION, STORMS & CRASH RECOVERY

Guard Against Duplicates, Outage Storms, and Mid-Run Crashes

One transactional coordination store gives the worker a lock, an expiring lease, a fencing check, and a storm breaker.

WHERE WE ARE

Because queues deliver at least once and CI retries webhooks, two workers can pull the same bug at the same time—or a broken base image can break twenty repositories at once. *Figure 4* adds a deterministic **Dispatcher / Guard** backed by a transactional

Coordination store & kill switch. Three related safeguards on the case row prevent race conditions: an atomic **lock** (only one worker claims the case), an expiring **lease** renewed by heartbeats (so a crashed worker's claim expires after 120 seconds instead of staying stuck forever), and a monotonic **fencing** generation number checked right before writing (so if Worker A stalls past 120 seconds and Worker B takes over at generation 2, Worker A is blocked from writing late).

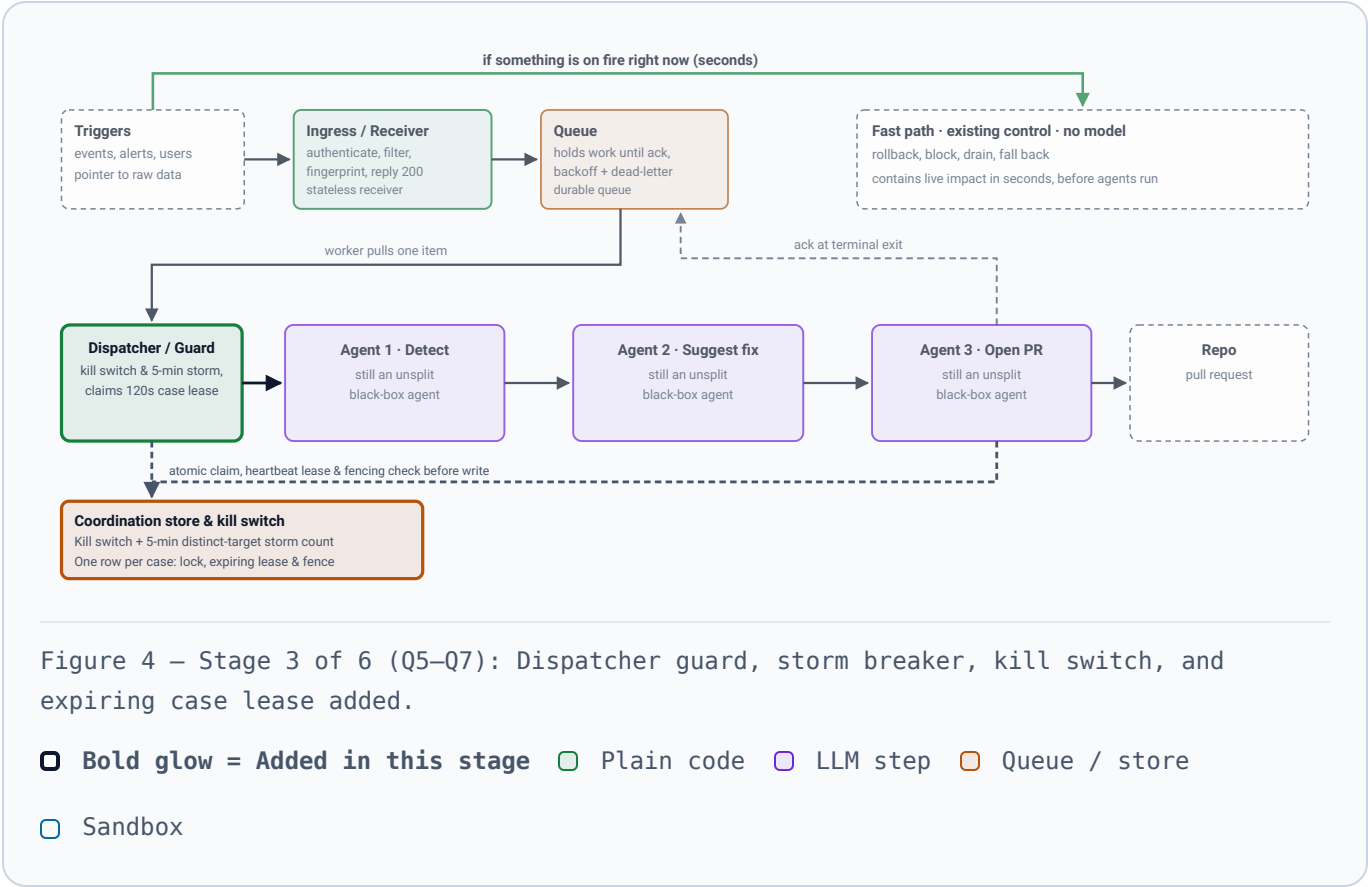


Figure 4 — Stage 3 of 6 (Q5–Q7): Dispatcher guard, storm breaker, kill switch, and expiring case lease added.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

5 How do I avoid doing the same thing twice?

SKIP IT AND…

- Fifteen identical pull requests, tickets, or refunds open for one bug.
- A retry repeats an external write that already succeeded.

DECIDE

- Compute a deterministic duplicate key in plain code from stable fields—never by a model.
- Keep one durable row per key (`IN_PROGRESS`, `DONE`, `ESCALATED`) and pass an idempotency key on external writes.

IN THE SELF-HEALING REPO

- `failure_fingerprint`: hash of error type, code location, and cleaned message.
- One row per `(repo, failure_fingerprint)`; finished rows immediately `ack` and skip.

6 How do I tell one problem from everything being on fire?

SKIP IT AND…

- A shared registry or compiler outage makes workers "fix" twenty healthy repositories.
- Token budget burns and engineers get flooded with bad PRs.

DECIDE

- Count *distinct* targets failing over a short sliding window.
- Above a threshold, trip a circuit breaker, pause automated writes, and page a human once; provide a manual kill switch.

IN THE SELF-HEALING REPO

- 5-minute query of distinct failing repos in the coordination database.
- 5 or more repos failing → pause PR creation for 15 minutes and page on-call.

7 What happens when a worker dies halfway?

SKIP IT AND…

- `ack` on receipt: in-flight work vanishes when a worker crashes.
- Permanent lock without expiry: the case stays stuck in `IN_PROGRESS` forever.

DECIDE

- `ack` only after a terminal state is recorded.
- Claim the row with an expiring **lease** renewed on a heartbeat, and verify the lease generation (**fencing**) right before any external write.

IN THE SELF-HEALING REPO

- 120-second lease renewed every 30 seconds; unacknowledged crashes redeliver via Pub/Sub.
- Lease generation checked right before opening the draft PR.

Added in Stage 3: Dispatcher / Guard + Coordination store (lock, expiring lease, fencing, storm breaker, kill switch)

Still open: The three agents are still unexamined purple black boxes

The plumbing around the worker is now reliable. Stage 4 opens up the three purple boxes themselves and asks which ones should be models at all.

STAGE 4 OF 6 · QUESTION 8 · AUTONOMY SPLIT, DUAL-LLM & HARNESS

Open the Black Boxes: Dual-LLM Separation, Verifier, and Plain-Code Gate

Give every step the lowest autonomy that works, separate untrusted reading from tool use, and wire control flow in code.

WHERE WE ARE

Up to *Figure 4*, all three scenario agents were drawn as identical purple boxes. Question 8 breaks that illusion in *Figure 5*. Inside the deterministic worker (**the harness**—the plain-code controller around the models), the two genuine model steps follow the **Dual-LLM pattern** (Willison; formalized with capability enforcement in CaMeL, and examined for ADK session-history isolation in Episode 01): Agent 1 becomes a toolless **Quarantined LLM** that

turns untrusted log text into a fixed schema, while Agent 2 becomes the **Privileged LLM** permitted to invoke workspace tools in a bounded loop. Between Agent 2 and Agent 3 we insert a deterministic **Verifier** so the model never grades its own work, and Agent 3 turns green as a plain-code **Act gate**—because opening a pull request follows a fixed rule and needs no language model at all.

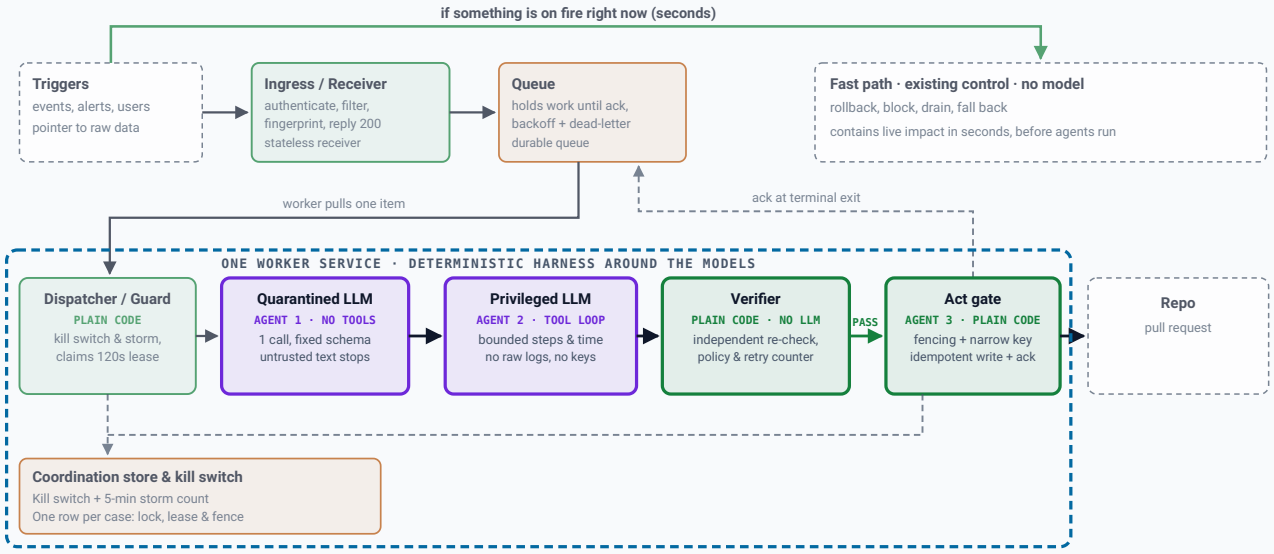


Figure 5 — Stage 4 of 6 (Q8): Black boxes split into Quarantined LLM, Privileged LLM, plain-code Verifier, and Act gate.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

8 Which steps need a model, which are code, and what is the harness?

SKIP IT AND…

- "Three models chatting": unpredictable routing, unbounded loops, impossible to audit.
- A model grades its own patch and agrees with its own mistakes.
- A model holds repository write tokens just to call a deterministic REST endpoint.

DECIDE

- Apply the four-row autonomy test (*Table 1*). If each of ten model steps is right 95% of the time, all ten succeed together ~60% of the time.
- Split untrusted reading (**Quarantined LLM**, zero tools) from tool calling (**Privileged LLM**, bounded loop).
- Wrap both in a deterministic **harness** that owns routing, retry counters, verification, and credentials.

IN THE SELF-HEALING REPO

- **Agent 1 (Quarantined LLM):** 1 schema-bound call over ≤100 screened lines, no tools.
- **Agent 2 (Privileged LLM):** bounded tool loop with step and time caps.
- **Verifier & Agent 3 (Act gate):** plain deterministic code, zero LLM calls.

TABLE 1 · FOUR-ROW AUTONOMY TEST — CHOOSING PLAIN CODE VS. MODEL STEPS

IF THE STEP…	BUILD IT AS	DETERMINISTIC BOUNDARY	IN THE SELF-HEALING REPO
follows an explicit rule	Plain code	None needed—same input, same output.	Filter, fingerprint, storm check, path rules, retry counter, opening the draft PR (Agent 3)
must interpret messy text	Quarantined LLM (one call)	Zero tools, `include_contents="none"`, fixed output schema, bounded input.	Agent 1 turns ≤100 screened error lines into an `IncidentReport`
must explore to find an answer	Privileged LLM loop + code verifier	Step cap, time cap, isolated tool sandbox. Plain code re-verifies the output.	Agent 2 searches, edits, and tests; Verifier re-runs full suite in a clean copy
mutates the outside world	Plain code + human gate	Fencing check, narrow permission, idempotency key, human review.	Act gate opens a draft PR on `autofix/*` and sends terminal `ack`

How the ADK Workflow Sketch Works

To enforce the structure in *Figure 5* rather than letting agents hand off to one another in free-form prose, the sketch below wires the worker as an explicit ADK 2.x `Workflow`

graph (`self_healing_workflow`). ADK's `edges` list mixes plain Python functions and `LlmAgent` nodes in one deterministic state machine: `"START"` runs `dedup_and_circuit_breaker` (the Guard), whose returned route selects `detector_agent` (`"ANALYZE"`), `skip_node` (`"SKIP"`), or `escalate_node` (`"ESCALATE"`). When analysis runs, `detector_agent` (Quarantined LLM) passes a validated `IncidentReport` to `route_incident` , a plain function that diverts `"flaky"` or `"unknown"` reports straight to `escalate_node` and routes actionable bugs (`"FIX"`) to `fixer_coding_agent` (Privileged LLM), which hands a typed `ProposedPatch` to `verify_patch_in_sandbox` .

Inside `verify_patch_in_sandbox(ctx, patch)` , plain code owns both policy enforcement and loop termination. First, `touches_protected_path` blocks edits to sensitive paths and immediately returns `Event(route="ESCALATE")` . Next, `run_sandbox_tests` tests the diff in a fresh container with `--network=none` and zero credentials. On a pass, `Event(route="PASS")` forwards the verified patch to `open_draft_pr` (Agent 3). On a failure, code increments `ctx.state["fix_attempts"]` : while fewer than three attempts have run, `Event(route="RETRY")` sends at most `2000` characters of `stderr` back to `fixer_coding_agent` ; on the third failure, `Event(route="ESCALATE")` guarantees termination.

```
The same graph sketched with ADK 2.x Workflow (agent and helper definitions omitted)

from google.adk import Agent, Context, Event, Workflow

def verify_patch_in_sandbox(ctx: Context, patch: ProposedPatch) -> Event:
    if touches_protected_path(patch.target_files):
        return Event(route="ESCALATE", output={"reason": "protected_path"})
    result = run_sandbox_tests(patch.unified_diff)  # --network=none, 0 creds
    if result["passed"]:
        return Event(
            route="PASS",
            output={"patch": patch, "stdout": result["stdout"]},
        )
    attempts = ctx.state.get("fix_attempts", 0) + 1
    ctx.state["fix_attempts"] = attempts
    if attempts < 3:
        return Event(route="RETRY", output={"stderr": result["stderr"][:2000]})
    return Event(route="ESCALATE", output={"reason": "max_attempts_exhausted"})

self_healing_workflow = Workflow(
    name="self_healing_code_repo",
    edges=[
        ("START", dedup_and_circuit_breaker),
        (dedup_and_circuit_breaker, {
            "ANALYZE": detector_agent,
            "SKIP": skip_node,
            "ESCALATE": escalate_node,
        }),
        (detector_agent, route_incident),              # typed IncidentReport handoff
        (route_incident, {"FIX": fixer_coding_agent, "ESCALATE": escalate_node}),
        (fixer_coding_agent, verify_patch_in_sandbox), # typed ProposedPatch handoff
        (verify_patch_in_sandbox, {
            "PASS": open_draft_pr,                    # Agent 3: plain code + PR token
            "RETRY": fixer_coding_agent,              # code-owned back-edge
            "ESCALATE": escalate_node,
        }),
    ],
)
```

Added in Stage 4: Dual-LLM split (Quarantined + Privileged LLM) + plain-code Verifier + plain-code Act gate inside one Workflow harness

Still open: Exact handoff payloads, retry/escalation paths on the diagram, session & long-term memory hygiene, sandboxes

Now that each node has the right autonomy, Stage 5 defines what crosses the arrows between them—and who is allowed to write state and memory.

STAGE 5 OF 6 · QUESTIONS 9–10 · TYPED HANDOFFS, RETRIES & STATE

Define Typed Handover Artifacts, Bounded Retries, and Memory Ownership

Pass small validated schemas instead of chat history, cap retries, escalate cleanly, and keep models from writing long-term memory.

WHERE WE ARE

Separating the Quarantined LLM from the Privileged LLM fails if the runtime silently forwards shared conversation history—including raw untrusted logs—into Step 2 (Episode 01), or if a failed patch loops forever. *Figure 6* lights up the four typed handover artifacts (`→ FailureEvent`, `→ IncidentReport`, `→ ProposedPatch`, `→ Draft PR`), adds the bounded `RETRY (< N)` back-edge and `ESCALATE to person` exit, and introduces per-run **Session**, pointer-addressed **blobs**, and read-only **long-term memory** of human-approved cases.

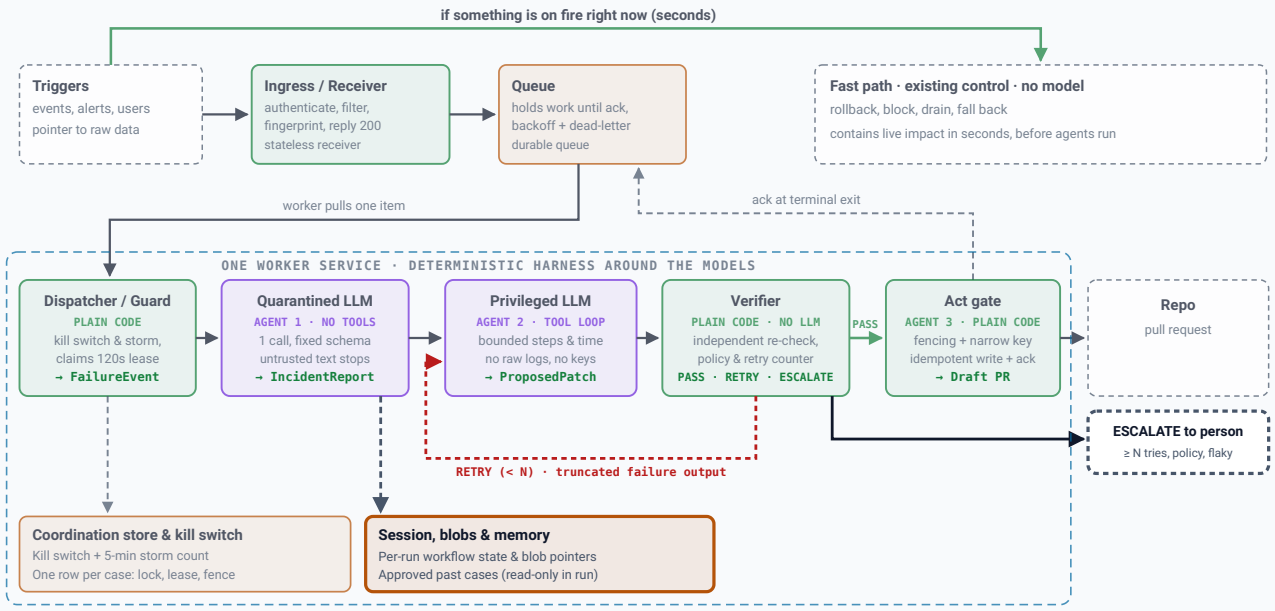


Figure 6 — Stage 5 of 6 (Q9–Q10): Typed handoff artifacts, bounded retry (< N), human escalation, and session/memory stores.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

9 What passes between the agents, and who decides what is next?

SKIP IT AND…

- Each step reinterprets the previous model's prose; errors compound.
- Shared session history leaks raw untrusted text past the Quarantined LLM into the Privileged LLM.
- A retry loop has no exit.

DECIDE

- Pass small typed contracts validated in code—never raw chat history (set `include_contents="none"`).
- Keep routing (PASS, RETRY, ESCALATE) and retry counters in deterministic code.
- Give the classifier an explicit abstain exit (`flaky` / `unknown` → escalate).

IN THE SELF-HEALING REPO

- `FailureEvent` → `IncidentReport` → `ProposedPatch` → `Draft PR`.
- Verifier caps retries at `N = 3`; `flaky` or `unknown` skips patching and escalates.

10 Where does the data live, and how does the agent read it?

SKIP IT AND…

- A pre-built search index lags behind the failing commit or drops permissions.
- Whole files and 50 MB stack traces flood the model's context window.
- Unverified model guesses get saved into shared long-term memory and poison future runs.

DECIDE

- Read data in place by pointer with strict bounds (exact version, time window, line range).
- Keep per-run step state in a session store and large payloads in object storage.
- Make cross-run long-term memory **read-only** during a run; update it in code only after human approval.

IN THE SELF-HEALING REPO

- Check out the exact failing `commit_sha`; bound tools (`ripgrep_search`, `read_file_lines`) and cap logs at 100 lines.
- Long-term memory holds only past human-merged fixes, written post-merge by code.

Typed Handoffs on a Concrete Failure

Suppose commit `a3f91c2` in `acme/payments` starts failing on a canary with `KeyError: 'vat_rate'` in `payments/tax.py` because a tax rate is missing for one country. Workflow code validates each schema on exit and entry—these are the exact payloads that cross the boundaries:

RECEIVER → WORKER

FailureEvent

```
{
  "repo":
"acme/payments",
  "commit_sha":
"a3f91c2",
  "failure_fingerprint":
"fp_8d4e",
  "log_pointer":
"dep_4471"
}
```

A pointer to the logs, not the 50 MB log payload.

AGENT 1 → AGENT 2

IncidentReport

```
{
  "category":
"test_regression",
  "suspected_files":
["payments/tax.py"],
  "failing_test":
"tests/test_tax.py",
  "confidence": 0.86
}
```

Raw logs stop here. Agent 2 never sees log text or Agent 1's prompt.

AGENT 2 → VERIFIER → AGENT 3

ProposedPatch

```
{
  "target_files":
["payments/tax.py"],
  "unified_diff": "---
a/… +++ b/…",
  "explanation": "Default
missing vat_rate"
}
```

Agent 3 receives it only after the verifier's own test run passes.

Who Owns Which State

A single store cannot serve every state need in an agentic workflow: cross-worker locks and storm limits require atomic database transactions, per-run step history and raw blobs belong to one execution, and long-term memory across runs must never be written by an unverified model loop. *Table 2* separates state by store and writer so one bad run or injected instruction cannot poison future runs.

TABLE 2 · STATE OWNERSHIP ACROSS COORDINATION, EXECUTION, MEMORY, AND AUDIT

STATE	WHERE IT LIVES	WHO WRITES IT	WHY
Kill switch & 5-minute storm count	Transactional coordination DB (`Spanner`)	Workflow code / operators	Trips the circuit breaker when 5+ distinct repos fail in 5 minutes, or pauses all workers when an operator flips the kill switch (no separate Redis cache needed at ~100 failures/day).
Incident lock, expiring lease & fencing	Coordination DB row per `(repo, failure_fingerprint)`	Workflow code only	**Lock:** one active owner. **Lease:** expires after 120s unless renewed every 30s so crashed work recovers. **Fencing:** lease generation checked right before PR creation so a stalled worker cannot write late.
Per-run workflow state & raw blobs	Session store + Object storage (`Agent Platform Session` + `Cloud Storage`)	ADK runner / workflow code	Carries typed outputs and `fix_attempts` across `RETRY` while keeping raw logs and diffs out of prompts by pointer.
Sandbox working scratchpad	Ephemeral container filesystem (`Sandbox 1`)	Privileged LLM tools (ephemeral)	Holds the checked-out commit, trial edits, and local test output for one attempt; destroyed when the attempt ends.
Long-term memory of approved fixes	Managed memory store (`Memory Bank`)	Post-merge code only (read-only to Agent 2)	Lets Agent 2 look up lessons from fixes a human engineer already reviewed and merged; never written during an unverified run.
Audit trace & eval replay store	Trace & analytics store (`Cloud Trace` + `BigQuery` + `ADK Eval`)	Harness on every call	Records every model turn, tool call, latency, and verifier verdict for incident audit, offline replay, and online quality tracking.

```
Added in Stage 5: Typed handover artifacts, include_contents="none", bounded RETRY (< N), ESCALATE, and separated state/memory ownership

Still open: Untrusted code execution isolation, narrow write permissions, human sign-off, and staged rollout
```

One danger remains: Agent 2 and the Verifier execute repository code and candidate patches, and the system still needs a safe way to ship and measure quality. Stage 6

completes the generic architecture.

STAGE 6 OF 6 · QUESTIONS 11–12 · SECURITY ISOLATION, ROLLOUT & COMPLETED SKELETON

Isolate Code Execution, Gate Writes, and Measure Rollout

Put untrusted execution in credential-free sandboxes, keep a human on irreversible writes, and gate every change with offline and online evals.

WHERE WE ARE

Running repository tests or model-written diffs inside the worker container would expose cloud identity and repository tokens to arbitrary code execution, and launching without replay metrics leaves engineers guessing whether the bot helps or creates noise. Adding isolated sandboxes (`Sandbox 1` and `Sandbox 2`), a human review loop, and the **Audit and eval store** in *Figure 7* completes the reusable generic architecture.

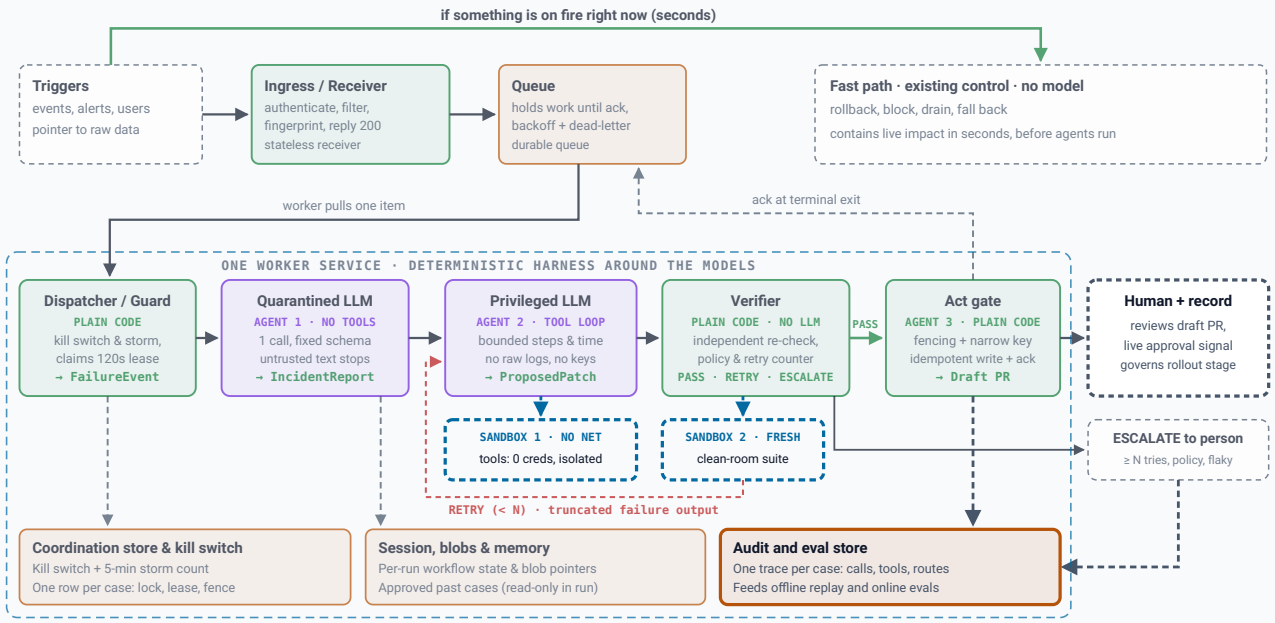


Figure 7 — Stage 6 of 6 (Q11–Q12): Isolated sandboxes, human gate, and audit/eval store complete the generic architecture.

Bold glow = Added in this stage Plain code LLM step Queue / store Sandbox

11 What would an attacker do with it?

SKIP IT AND…

- Injected log text steers a tool-calling model.
- Malicious test code or a crafted diff steals cloud IAM or GitHub tokens from the worker host.
- Over-broad write scopes allow pushes to `main` or edits to CI workflows.

DECIDE

- Keep the **Quarantined LLM** toolless (Dual-LLM / CaMeL) and sever shared history.
- Execute the **Privileged LLM**'s tools and the **Verifier**'s test suite inside ephemeral sandboxes with zero network and zero credentials.
- Enforce protected-path rules and narrow write scopes in code; keep a human on irreversible actions.

IN THE SELF-HEALING REPO

- HMAC check, content screening, and toolless Agent 1.
- `Sandbox 1` (Agent 2 tools) and fresh `Sandbox 2` (Verifier pytest) with `--network=none` and 0 credentials.
- Protected-path blocklist, token restricted to `autofix/*` branches, draft PRs only.

12 How do I switch it on, and how do I know it is working?

SKIP IT AND…

- A noisy bot opens broken PRs and gets muted by engineers within a week.
- Prompt or model upgrades silently regress accuracy with no safety net.

DECIDE

- **Offline evals:** gate every prompt, tool, or model change against a golden replay set (tracking pass@1 and pass@3).
- **Online rollout:** progress through stages (shadow → draft → ready-for-review) with automatic demotion on human rejection spikes.
- Record one end-to-end trace per case across every call and verdict.

IN THE SELF-HEALING REPO

- Replay historical repository bugs before every release.
- Shadow mode → draft PRs → ready-for-review; track merge-without-edit rate and human diff-edit distance.

Every Component of the Completed Architecture

Table 3 summarizes all thirteen components assembled in *Figure 7*—stating the domain-general failure mode and mechanism first, then how each maps to the self-healing repository system. *Table 4* shows how the exact same skeleton transfers to two other domains with only the nouns changed.

TABLE 3 · GENERIC COMPONENT BREAKDOWN OF FIGURE 7 & SELF-HEALING REPO MAPPING

COMPONENT	KIND	PROBLEM IT SOLVES	HOW IT WORKS (GENERIC)	IN THE SELF-HEALING REPO
Triggers	External	Signals arrive from separate systems in incompatible formats.	Emits raw webhook, alert, or user events whenever an upstream condition fires.	CI failure, canary rollout alert, or monitoring alarm across 20–30 repositories.
Ingress / Receiver	Plain code	Senders time out on slow runs, and open endpoints accept spoofed noise.	Authenticates sender, drops noise, hashes duplicate key, enqueues pointer event, replies `200`.	Cloud Run receiver verifies HMAC/OIDC, hashes `failure_fingerprint`, publishes `FailureEvent`.
Fast path · existing control	External, no model	Live impact cannot wait minutes for an agent to reason and test.	Existing deterministic automation contains blast radius in seconds while agents run asynchronously.	Deployment controller rolls back the failing canary in seconds without calling a model.

COMPONENT	KIND	PROBLEM IT SOLVES	HOW IT WORKS (GENERIC)	IN THE SELF-HEALING REPO
Queue	Queue	Bursts drop events, and crashes lose work if items are deleted on receipt.	Buffers items, retries with backoff + DLQ, and deletes an item only on terminal `ack`.	Pub/Sub holds `FailureEvent` until `PR_OPENED`, `SKIP`, or `ESCALATED`; DLQ after 5 crashes.
Dispatcher / Guard	Plain code	Duplicate events spawn duplicate runs; shared outages flood every target.	Checks kill switch and short-window storm count, then atomically claims a case lease.	Pauses if 5+ repos fail in 5 min; skips finished/live rows; claims 120s lease in Spanner.
Quarantined LLM	LLM (1 call, 0 tools)	Untrusted external text can carry prompt injections aimed at tools.	Single toolless call (`include_contents="none"`) emitting a strict schema; raw text stops here.	**Agent 1 · Detect:** ≤100 screened log lines → `IncidentReport` (or `flaky` / `unknown` → escalate).
Privileged LLM + Sandbox 1	LLM loop + sandbox	Open-ended repair requires iterative exploration, yet untrusted code risks credential theft.	Step/time-capped loop whose tools run in a container with zero network and zero credentials.	**Agent 2 · Suggest fix:** search, read, patch, and local test in Sandbox 1 → `ProposedPatch`.
Verifier + Sandbox 2	Plain code + sandbox	Models misgrade their own work and can game checks by deleting tests.	Re-verifies candidate output in a fresh sandbox against domain & policy rules; caps retries.	Fresh Sandbox 2 re-runs full pytest, blocks protected paths/deleted tests, caps `N = 3`.
Act gate	Plain code	Models must not hold write keys, and stalled workers can write late.	Runs lease fencing check, performs narrowest idempotent write, and sends terminal `ack`.	**Agent 3 · Open PR:** checks Spanner lease epoch, opens draft PR on `autofix/*` + Pub/Sub `ack`.
Human + record	Human	Irreversible changes need human accountability and live quality telemetry.	Reviewer approves, edits, or rejects; live acceptance rate gates rollout autonomy.	Code owner reviews draft PR; merge-without-edit rate governs staged promotion/demotion.
Coordination store	Transactional DB	Workers race on duplicates, crash mid-run, or stall past lease expiry.	Holds kill switch, storm query, and case row with atomic **lock**, expiring **lease**, and **fencing**.	Single Spanner database (control row + `incidents` table with 120s lease & 30s heartbeat).
Session, blobs & memory	Session, blob & memory	Large blobs bloat prompts; unverified model writes poison cross-run memory.	Per-run state in session, raw blobs by pointer, read-only memory written post-approval by code.	`fix_attempts` in Session, logs/diffs in Cloud Storage, human-merged fixes in Memory Bank.
Audit and eval store	Trace & analytics	Without per-case traces, regressions cannot be audited or replayed offline.	Logs every model call, tool run, latency, and verdict for offline replay and online metrics.	Cloud Trace + BigQuery + ADK Eval golden replay set of historical bugs.

TABLE 4 · APPLYING THE SAME ARCHITECTURE ACROSS THREE DOMAINS

	SELF-HEALING REPO	WEATHER ALERT → REROUTE SHIPMENTS	RISK REVIEW OF 100,000 BANK CLIENTS
Wakes it up	CI, CD controller, monitoring	Weather alert feed, carrier status	Nightly feed of clients whose data changed
Real size	100 failures a day	Few alerts, each touching thousands of shipments	Only the changed clients, not all 100,000
Fast, no model	Rollback by the CD controller	Rule-based hold on departures in the alert area	None needed; batch workload
Duplicate key	`(repo, failure_fingerprint)`	`(alert_id, shipment_id)`	`(client_id, review_period)`
Everything on fire?	5 repos fail in 5 minutes	Broken feed flags every region at once	Broken data feed flags thousands at once
Quarantined / Privileged LLM	Classify logs; iteratively patch & test	Parse alert; explore reroute options	Extract filing facts; draft risk narrative
Code steps	Verify suite; path rules; open draft PR	Cost/capacity rules; idempotent booking	Financial ratios in code; tier routing
Human	Reviews and merges the draft PR	Approves reroutes above a cost threshold	Signs off by risk tier

Added in Stage 6: Ephemeral Sandboxes 1 & 2 + Human rollout gate + Audit & eval store

Generic architecture complete — ready to map onto cloud services

CLOUD IMPLEMENTATION · GOOGLE CLOUD & AGENT PLATFORM

Mapping the Assembled Architecture to Google Cloud

A 1-to-1 mapping onto Google Cloud and the Gemini Enterprise Agent Platform, and when a node becomes its own service.

Figure 8 maps every box of *Figure 7* onto Google Cloud and the **Gemini Enterprise Agent Platform** (shortened to **Agent Platform** below; LLM calls remain model-family neutral). At ~100 failures a day, one **Spanner** database covers the kill switch, the 5-minute distinct-repo storm query, and the per-incident lock, expiring lease, and fencing generation check—

an extra in-memory cache like Memorystore is unnecessary unless event rates reach thousands per second. Alongside Spanner, Agent Platform provides managed per-run **Session** storage, **Memory Bank** for read-only lookup of past human-merged fixes, ephemeral gVisor sandboxes via **Agent Platform Code Execution**, keyless **Agent Identity**, **Agent Gateway** with **Model Armor**, and **ADK Eval**.

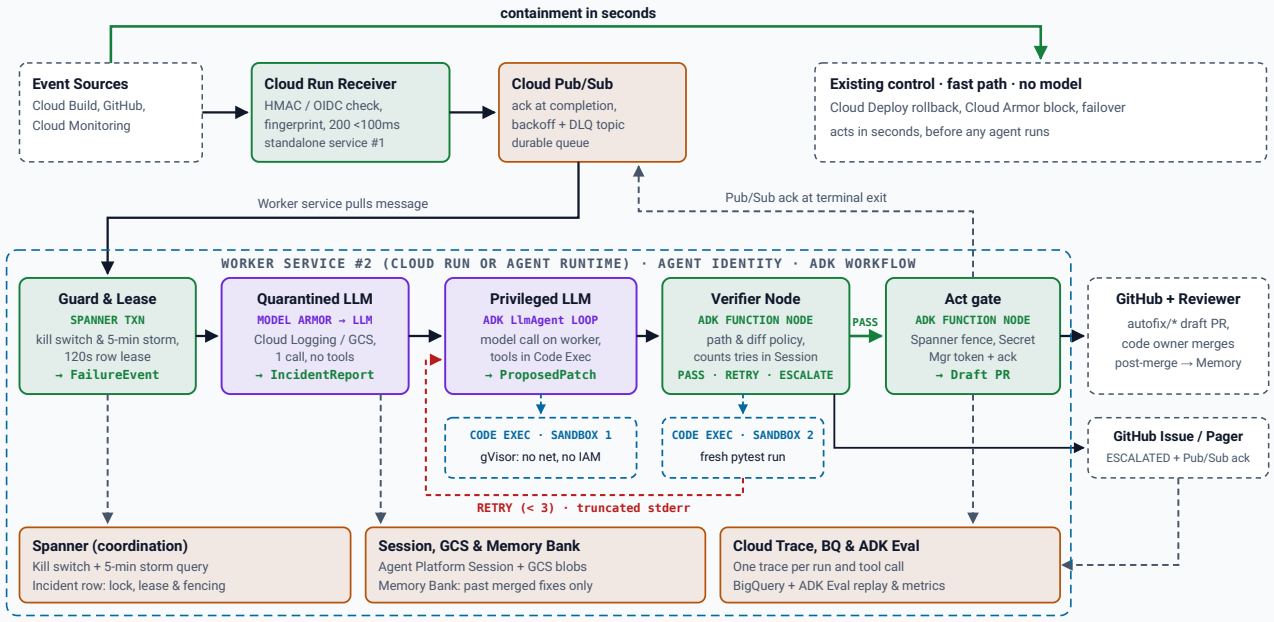


Figure 8 — The assembled architecture from *Figure 7* mapped 1-to-1 onto Google Cloud and Agent Platform.

Plain code LLM step Queue / store Sandbox

What Gets Deployed as Its Own Service

Drawing several steps in a workflow graph often tempts teams to deploy every box—especially every agent—as a separate microservice over HTTP. Turning sequential steps of the same task into separate network services adds extra hops, serialization bugs, authentication policies, and partial-failure modes for zero gain. By default, workflow nodes—both `LlmAgent` steps and plain functions—run **in-process inside a single worker service**, where one queue message drives one complete run of the graph.

Split a step out into its own standalone deployment or runtime boundary only when one of four conditions holds:

- **Different ingress or latency contract:** a public endpoint that must answer callers in milliseconds cannot block on a multi-minute agent run.
- **Security and isolation boundary:** untrusted code or test suites must not execute inside the container that holds cloud identity or external write tokens.
- **Independent scaling or hardware:** one step needs GPUs, large memory, or scales at a completely different rate than the rest of the graph.
- **Separate team ownership:** a capability is owned and released independently by another team behind an explicit API or tool contract.

Applying those four criteria produces the exact two-service layout in *Figure 8*: a lightweight **Cloud Run Receiver** at the public webhook edge, and one **Worker Service** (on Cloud Run or Agent Runtime) running all workflow nodes in-process—dispatching only untrusted tool and test execution across an isolation boundary into ephemeral, credential-free **Code Execution** sandboxes while keeping model calls and the pull-request token on the worker.

SUMMARY · MASTER CHEAT SHEET

Twelve-Question Design Cheat Sheet

Every question in execution order, what breaks if you skip it, and what it adds to the diagram.

HYBRID ARCHITECTURE RULE

Non-deterministic model steps generalize over messy inputs and explore candidate fixes that static rules cannot anticipate; deterministic controller code owns the workflow, verifies every result independently, and gates state, writes, and queue acknowledgement.

TABLE 5 · MASTER CHEAT SHEET — ALL TWELVE QUESTIONS MAPPED TO PROGRESSIVE STAGES

Q#	DESIGN QUESTION	WHAT BREAKS WITHOUT IT	ARCHITECTURAL RULE / BOX ADDED	SELF-HEALING REPO	STAGE
Q1	What wakes it up & what is in the message?	Missed triggers or 50 MB log payloads on the bus.	Normalize every trigger into one small event carrying a data **pointer**.	CI, canary, alerts → `FailureEvent`	Stage 1
Q2	How big is it really?	Over-built infra for filtered noise, or runaway model bills.	Multiply volume × filter rate; enforce step and budget caps per case.	`10,000 × 1% = 100` /day	Stage 1
Q3	What must happen in seconds?	Users bleed during slow LLM runs, or models push unreviewed hotfixes.	Route urgent containment to an existing **model-free fast path**.	CD controller rolls back canary	Stage 1
Q4	How does work reach a worker safely?	Webhook timeouts, dropped bursts, or lost crashes.	Stateless **Receiver** + durable **Queue** + terminal `ack` + DLQ.	Cloud Run → Pub/Sub → terminal `ack`	Stage 2
Q5	How do I avoid doing work twice?	Duplicate PRs, tickets, or double mutations.	Code-computed duplicate key + atomic **lock** row + idempotent write.	`(repo, failure_fingerprint)` in Spanner	Stage 3
Q6	One bug vs. everything on fire?	Shared infra outage makes bots "fix" healthy targets.	Count distinct targets in a short window; trip **storm breaker** + kill switch.	5+ repos in 5 min → pause & page	Stage 3
Q7	What if a worker dies halfway?	Lost work (early `ack`) or permanently stuck locks.	Expiring **lease** + heartbeat renewal + pre-write **fencing** check.	120s lease, 30s heartbeat, epoch fence	Stage 3
Q8	Model or code, and what is the harness?	Models chatting unboundedly, self-grading, or holding write keys.	Dual-LLM split (**Quarantined** vs. **Privileged LLM**) + code **Verifier** & **Act gate**.	ADK `Workflow` graph (*Table 1*)	Stage 4
Q9	What passes between steps & who routes?	Prose drift, prompt-injection history bleed, infinite loops.	Typed schemas, `include_contents="none"`, capped `RETRY (< N)`, `ESCALATE`.	`IncidentReport` → `ProposedPatch`, cap 3	Stage 5
Q10	Where does data live & who writes state?	Stale indexes, context bloat, poisoned long-term memory.	Pointer reads at exact version; code-owned state; post-approval memory only.	Exact `commit_sha`; post-merge Memory Bank	Stage 5
Q11	What would an attacker do with it?	Prompt injection → RCE or stolen cloud/repo tokens.	Toolless Q-LLM, no-net/no-cred **Sandboxes 1 & 2**, narrow write scope + human.	Code Exec gVisor, `autofix/*` draft PR	Stage 6
Q12	How do I launch & prove it works?	Noisy bot muted in week one; silent prompt regressions.	**Audit & eval store**: offline golden replay + staged rollout + auto-demotion.	Shadow → draft PR; merge-without-edit rate	Stage 6

Pacing a 20-minute design session: spend the first 3 minutes on Stage 1 (Q1–Q3: triggers, real scale, fast containment), 5 minutes sketching Stages 2–3 (Q4–Q7: intake queue, terminal `ack`, storm breaker, lease + fencing), 8 minutes on Stages 4–5 (Q8–Q10: Dual-LLM split, plain-code Verifier and Act gate, typed handoffs, memory ownership), and the final 4 minutes on Stage 6 (Q11–Q12: sandboxes, narrow write scope, replay evals, and staged rollout).

SERIES INTEGRATION · COMPLETING THE PICTURE

How This Blueprint Fits Into Episode 01

Episode 02 designs the outer workflow topology; Episode 01 zooms deep into the enterprise data and tool wire inside it.

In the self-healing repository example, the Privileged LLM's tools operated on a checked-out git commit inside an air-gapped sandbox, and the Act gate called GitHub's pull-request API. In most enterprise systems—customer support, financial operations, supply-chain rerouting, clinical or HR workflows—the **Privileged LLM** and the **Act gate** instead read from and write to live corporate databases and internal APIs on behalf of a human caller.

That seam is exactly where Episode 01 — Securing the Agent-to-Database Connection plugs into *Figure 7*. Episode 02 governs the **macro control flow** (how events enter, when not to use a model, how multi-step handoffs, retries, leases, and verifiers work); Episode 01 zooms into the **data and tool wire** leaving the Privileged LLM and Act gate (*Figure 9* and *Table 6*) so that a prompt injection, confused deputy, or stolen token still cannot exfiltrate or corrupt multi-tenant enterprise data.

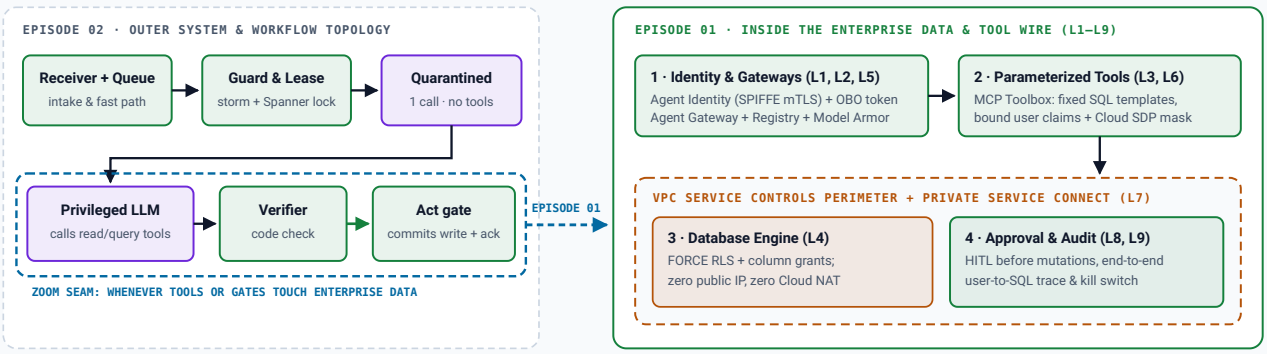


Figure 9 — Episode 02 designs the outer workflow and autonomy boundaries; Episode 01 locks down the enterprise tool and database wire inside it.

TABLE 6 · CONNECTING EPISODE 02'S WORKFLOW COMPONENTS TO EPISODE 01'S DEFENSE LAYERS (L1–L9)

EPISODE 02 COMPONENT	WHAT EPISODE 02 DECIDES	EPISODE 01 LAYER PLUGGED IN	WHAT EPISODE 01 ADDS ON THE DATA WIRE
Ingress / Receiver & Worker Identity	Verify external sender, authenticate statelessly, keep worker credentials off sandboxes.	**L1 · Ingress Proxy** & **L2 · Agent Identity (SPIFFE)**	Keyless SPIFFE workload certificates, mutual TLS, rate quotas, and Dual-Scope Authorization (`Agent Scope ∩ User Scope`).
Quarantined LLM & Typed Handoffs	Toolless schema extraction, `include_contents="none"` , bounded input lines.	**L6 · Inline Screening & DLP** + **L8 · Dual-LLM Quarantine**	Inline Model Armor prompt-injection filtering and Cloud SDP PII tokenization on both prompts and returned database text.
Privileged LLM Tool Calls	Bounded tool loop, narrow pointer-based tools, no raw query authoring when rules suffice.	**L3 · Parameterized Tool Server** & **L5 · Egress Proxy / OBO**	MCP Toolbox with pre-parameterized SQL templates (zero model-written SQL), Agent Registry allowlists, and RFC 8693 OBO user token exchange.
Coordination & Data Stores	Read data in place; enforce invariants in deterministic infrastructure rather than prompts.	**L4 · Database RLS / CLS** & **L7 · Private Link + VPC-SC**	Engine-enforced PostgreSQL `FORCE RLS` / column grants over Private Service Connect inside a VPC Service Controls data perimeter (zero public IP or NAT).
Act Gate, Human + Audit Store	Fencing check, narrow idempotent write, human review, per-case trace & kill switch.	**L8 · HITL Approval Gate** & **L9 · End-to-End Audit / Kill Switch**	Cryptographically bound human confirmation before mutations, principal-to-SQL audit join in BigQuery, and a 3-step emergency kill switch.

SOURCES

References

External literature on hybrid workflows, Dual-LLM security, and multi-agent failure modes.

- Cemri et al., Why Do Multi-Agent LLM Systems Fail? (arXiv:2503.13657). A taxonomy of specification, coordination, and verification failures across multi-agent traces.
- Anthropic, Building Effective Agents. Distinguishing deterministic workflows from open-ended agents, evaluator-optimizer loops, and tool interface design.
- Anthropic, How We Built Our Multi-Agent Research System. Effort scaling, context isolation across subagents, and production reliability lessons.
- Google Cloud, Choose a Design Pattern for Your Agentic AI System. Selecting single-agent, deterministic workflow, loop, or multi-agent patterns from task shape, latency, and budget.
- Cognition, Don't Build Multi-Agents. Why parallel writer swarms fail from fragmented context, and keeping writes single-threaded.
- HumanLayer, 12-Factor Agents. Engineering principles for production agents: owning your context window, structured tool outputs, unified state, and explicit control flow.
- Simon Willison, The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. Separating a toolless Quarantined LLM that reads untrusted input from a tool-equipped Privileged LLM mediated by a deterministic controller.
- Debenedetti et al., Defeating Prompt Injections by Design (CaMeL) (arXiv:2503.18813). Capability-tracked control and data flow between a Quarantined LLM and a Privileged LLM inside a deterministic interpreter.
- Episode 01 — Securing the Agent-to-Database Connection (including Dual-LLM shared-history isolation with `include_contents="none"`). The nine defense layers securing the enterprise tool and database wire inside this blueprint.